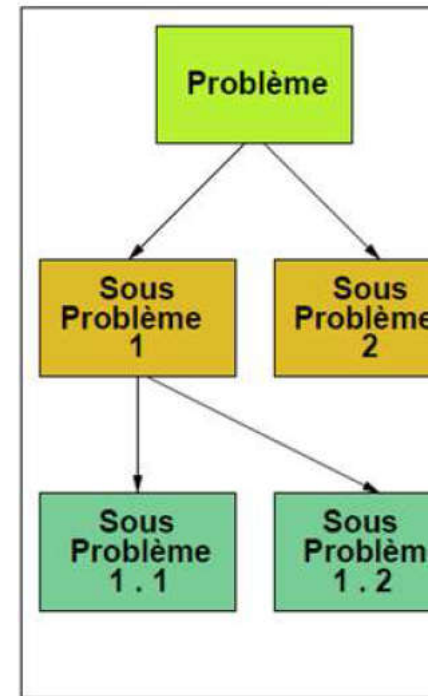


Chapitre 4 : Fonctions et procédures

Introduction

- Analyse descendante : Approche incrémentale consistant à analyser la spécification d'un problème selon son arborescence
- Décomposition / combinaison à chaque niveau de l'arborescence :
 - Décomposer le problème en un certain nombre de sous problèmes qu'on supposera résolus
 - Combiner les solutions des sous-problèmes pour résoudre le problème courant
 - Résoudre les sous-problèmes par la même approche



Introduction

Le but de l'utilisation de sous programmes

- Décomposition des problèmes en modules (sous problèmes de taille réduite)
 - Dans un programme plusieurs séquences d'instructions sont appelées plusieurs fois et depuis divers points du programme. Il serait donc plus intéressant d'isoler ces séquences dans un sous programme qui peut être appelé depuis n'importe quel point du programme
 - La lisibilité qui facilite notablement la compréhension du programme
- Réutilisation du sous programme

Introduction

- Un sous programme est une portion de code analogue à un programme
- Déclarée dans un programme ou dans un sous programme et dont la partie instruction peut être exécutée plusieurs fois au cours du traitement du programme grâce à des appels
- Un sous-programme est obligatoirement caractérisé par un nom (identifiant) unique
- Le programme qui utilise un sous-programme est appelé le programme appelant
- On distingue deux formes de sous programmes : **Les procédures et les fonctions**

Passage de paramètres

Passage de paramètres

Le processus de substitution des paramètres effectifs aux paramètres formels s'appelle passage de paramètres.

→ C'est un transfert d'informations entre le programme ou module appelant et le sous-algorithme appelé

Les 3 types de passage de paramètres :

Passage en entrée (par valeur)

Passage en sortie (par variable)

Passage en entrée/sortie (par variable)

Passage de paramètres

Passage de paramètre en entrée (par valeur)

Le transfert d'information s'effectue dans un seul sens :
l'algorithme appelant → sous-algorithme appelé

Fonctionnement :

Lors de l'appel, la valeur du paramètre effectif est copiée dans le paramètre formel du sous-algorithme

Le sous-algorithme **travaille sur une copie, pas sur l'original**

Conséquence :

Toute modification du paramètre formel n'affecte pas la variable d'origine

Le paramètre effectif reste inchangé

Utilisation typique :

Utilisé dans les fonctions et procédures lorsque **le sous-algorithme n'a pas à modifier la valeur transmise**

Passage de paramètres

Passage en sortie (par variable)

Le transfert d'information se fait dans le sens :
algorithme appelant $\leftarrow$ sous-algorithme appelé

Le sous-algorithme initialise ou modifie la variable

La valeur finale est renvoyée à l'algorithme appelant

Passage en entrée/sortie (par variable)

Le transfert d'information se fait dans les deux sens :
algorithme appelant $\rightleftarrows$ sous-algorithme appelé

Le sous-algorithme peut lire et modifier la valeur reçue

Toute modification du paramètre formel modifie directement le paramètre effectif

→ **On travaille sur la copie originale (par référence)**

Les fonctions

Une fonction est un sous programme qui renvoie une valeur d'un seul type

- Ce type sera celui de la fonction **type_valeur_retour_fonction** est le type du résultat renvoyé par la fonction
- Liste de paramètres est la liste des paramètres formels donnés en entrée avec leurs types de la forme $p_1: \text{type}_1, \dots, p_n: \text{type}_n$,
- Le corps de la fonction doit comporter une instruction de la forme :
`retourner(expression);`
- Cette instruction met fin à l'exécution de la fonction et retourne `expression` comme résultat

Les fonctions

Déclaration

Fonction *NomDeLaFonction (paramètres et leurs types) : type_valeur_retour_fonction*

Var *variable locale 1 : type 1; . . .*

début

instructions de la fonction

Retourner *expression*

FinFonction

- Entête de la fonction appelée aussi signature, prototype, ou déclaration de fonction
- Type_retour : Le type de retour d'une fonction doit avoir le même type que le résultat retourné; c'est un seul type qui peut être un entier, réel, booléen, caractère..
- Les parenthèses sont obligatoires même lorsqu'il n'y a pas de paramètres

Les fonctions

- Exemple

```
fonction surface(long : réel, larg : réel) : réel  
    Variables  
        surf : réel  
    surf ← long * larg  
    retourner surf  
fin fonction
```

Algorithme surfRect

Variables

aire, long, larg : réel

Début

Écrire("Longueur et largeur du rectangle")

Lire(long, larg)

aire ← surface(long, larg)

Écrire("La surface est : ", aire)

FinAlgorithme

Les fonctions

- Exemple

```
Fonction abs (x : entier) : entier
DEBUT
Si (x >= 0) alors
retourne x
Sinon
retourne -x
Finsi
FIN
```

```
Algorithme absolu
Variables
    a, res : Entier
Début
    Lire(a)
    res ← abs(a)
    Écrire(res)
FinAlgorithme
```

Les fonctions

- Exemple

crire une fonction qui renvoie la moyenne de deux entier.

fonction Moyenne(A : entier, B : entier) : réel

var : Moy : réel

Moy $\leftarrow (A + B) / 2$

Retourner Moy

FinFonction

Les fonctions

- Exemple

crire une fonction qui renvoie le maximum de deux réels.

```
fonction Maximum(A : réel, B : réel) : réel  
  Si (A ≥ B) Alors  
    Retourner A  
  Sinon  
    Retourner B  
  Fin Si  
Fin Fonction
```

Les fonctions – Appel d'une fonction

- L'appel d'une fonction se fait par simple écriture de son nom dans le programme principal suivi des paramètres entre parenthèses (les parenthèses sont toujours présentées même lorsqu'il n'y a pas de paramètre)
- L'appel de la fonction **retourne une valeur** calculée en fonction des **paramètres effectifs**
- L'appel d'une fonction se fait par simple écriture de son nom dans le programme principale. Le résultat étant une valeur, devra être affecté ou être utilisé dans une expression

Les fonctions

Trois façons d'appeler une fonction

. Dans une affectation

```
nom_variable ← Nom_fct({paramètres_réels})
```

. Dans un affichage

```
Écrire("Le résultat est : ", Nom_fct({paramètres_réels}))
```

. Dans une expression

```
x / 2 + Min(a, b)
```

Les fonctions

Exemple

```
x ← 3
y ← 5
z ← min(x, y)           // z = 3
z ← min(3, 5)           // z = 3
t ← (7 + min(x, y)) / 2 // t = (7 + 3) / 2 = 5
Écrire(min(x, y))        // Affiche : 3
Si min(x, y) > 0 Alors
    // Instructions...
FinSi
```


Les fonctions

- Exemple

Fonction Pair (n : entier) : booléen

Debut

retourner (n%2=0)

FinFonction

Algorithme AppelFonction

Var: c : réel, b : booléen

Début

b ← Pair(3)

.....

Fin

- Lors de l'appel Pair(3), le paramètre formel n est remplacé par le paramètre effectif 3

Les fonctions

Exemple

Comment utiliser la fonction min pour afficher le minimum de 3 entiers x, y et z en une seule instruction, sans utiliser de variables supplémentaires ?

- Solution

```
Écrire (min (min (x, y) , z) )
```

Les procédures – Déclaration

- Une procédure est un sous-programme semblable à une fonction mais **qui ne retourne rien**
- Une procédure s'écrit en dehors du programme principal sous la forme :

Procédure *NomDeLaProcédure (paramètres et leurs types)*

Var *variable locale 1 : type 1; . . .*

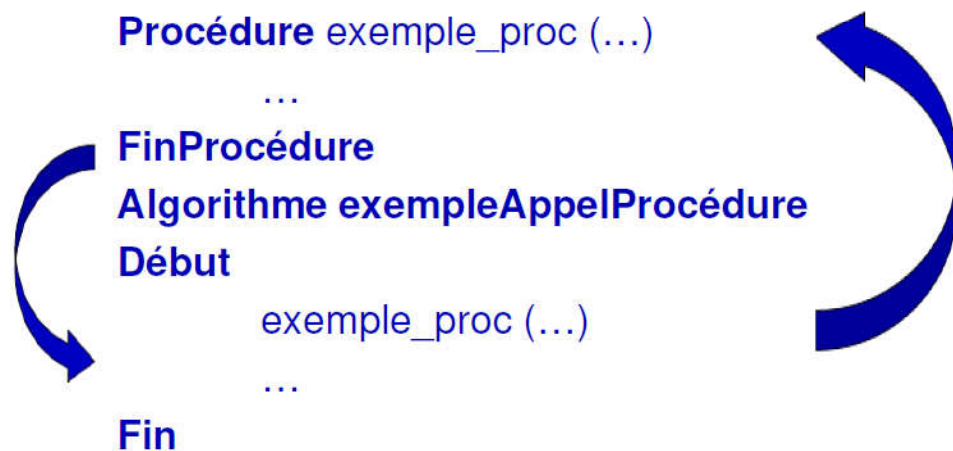
début

instructions de la procédure

FinProcédure

Les procédures – Appel

- L'appel d'une procédure, se fait dans le programme principal ou dans une autre procédure par une instruction indiquant le nom de la procédure :

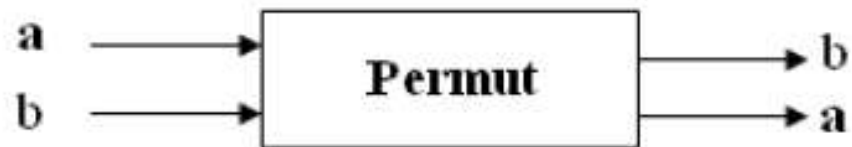


Remarque : contrairement à l'appel d'une fonction, on ne peut pas affecter la procédure appelée ou l'utiliser dans une expression

Les procédures – Appel

- Une procédure peut transmettre des données via des paramètres. Ces paramètres peuvent être passés de différentes manières :
 - **En entrée** : La procédure reçoit une valeur, mais ne la modifie pas.
 - **En sortie (Var)** : La procédure modifie la variable, qui est renvoyée à l'extérieur.
 - **En entrée/sortie (Var)** : La procédure reçoit une valeur, la modifie, et la renvoie.
- On appelle une procédure comme une fonction, en indiquant son nom suivi des paramètres entre parenthèses

Les procédures – Exemple



Procédure Permut(var a,b : Entier)

Var

aux : Entier

Début

aux $\leftarrow$ a

a $\leftarrow$ b

b $\leftarrow$ aux

Fin

Algorithme Test {programme principal}

Var

x,y : Entier

Début

Ecrire("Entrer un entier x="), Lire(x)

Ecrire("Entrer un entier y="), Lire(y)

Permut(x,y) //appel de la procédure

Ecrire("La nouvelle valeur de x=",x, " et de y=",y)

Fin.

Les procédures – Exemple

```
PROCEDURE PROC (A : entier ; var B, C : entier )
```

```
DEBUT
```

```
  A ← A - 1
```

```
  B ← A + 5
```

```
  C ← C * 3
```

```
  Ecrire (A, B, C)
```

```
FIN
```

```
ALGORITHME X
```

```
VAR N, P, Q : entier
```

```
DEBUT
```

```
  N ← 2
```

```
  P ← 2
```

```
  Q ← 2
```

```
  PROC (N, P, Q) → 1 6 6
```

```
  Ecrire (N, P, Q) → 2 6 6
```

```
FIN
```

Fonction - Procédure

- ❑ Une fonction rend un et un seul résultat



- ❑ Une procédure rend zéro ou plusieurs résultats



Fonction - Procédure

Fonction puissance1(a, b : entier) : entier

Var

i, c : entier

Début

c ← 1

Pour i de 1 à b faire

c ← c * a

FinPour

retourne c

FinFonction

Procédure puissance1(a, b : entier; Var c : entier)

Variabales

i : entier

Début

c ← 1

Pour i de 1 à b faire

c ← c * a

FinPour

FinProcédure

ALGORITHME X
VAR a,b,c: entier
DEBUT
 a ← 2
 b ← 3
 puissance (a,b,c)
FIN

exemple

Algorithme exemple

Variables
entier1, entier2, entier3, min, max : Entier

Fonction minimum2(a, b : Entier) : Entier

Variables
resultat : Entier

Si (a < b) Alors
 resultat ← a
Sinon
 resultat ← b
FinSi
Retourner resultat
FinFonction

Fonction minimum3(a, b, c : Entier) : Entier

Variables
resultat : Entier

resultat ← minimum2(minimum2(a, b), c)
Retourner resultat
FinFonction

Procédure calculerMinMax3(a, b, c : Entier ; Var min3
Début
 min3 ← minimum3(a, b, c)
FinProcédure

Début
 Écrire("Entrez trois entiers :")
 Lire(entier1)
 Lire(entier2)
 Lire(entier3)

 calculerMinMax3(entier1, entier2, entier3, min)

 Écrire("La valeur la plus petite est ", min)
FinAlgorithme

Types de variables manipulées

Variables globales :

- Déclarées dans l'algorithme principal
- Peuvent être utilisées directement aussi bien par l'algorithme principal que par tous les sous-algorithmes auxquels il fait référence
- Toute variable globale peut être utilisée par un sous-algorithme sans être déclarée dans celui-ci

Variables locales :

- Déclarées au niveau du sous-algorithme
- Sont visibles uniquement par ce sous-algorithme

Types de variables manipulées

Paramètres formels (ou fictifs) :

- Ce sont les paramètres qui se trouvent dans l'entête du sous-algorithme
- Alloués au moment de l'appel d'un sous-algorithme appelé
- Libérés lorsque le contrôle retourne au sous-algorithme appelant

Paramètres réels (ou effectifs) :

- Par lesquels l'algorithme appelant communique les valeurs nécessaires au sous-algorithme appelé
- Le sous-algorithme appelant copie le contenu des paramètres réels dans les paramètres formels correspondants
- Les paramètres réels doivent correspondre en nombre, en type et en ordre aux paramètres formels

Types de variables manipulées

Algorithme principal

Constantes $\pi = 3.14$

Var

x : réel

Variable globale

Sous-algorithme cercle (r : réel)

Paramètre formel

Var s : réel

Début

$s \leftarrow \pi * r * r$

Variable locale

écrire (" la surface est :", s)

Fin

Début

$x \leftarrow 22$

cercle (x)

Paramètre réel

Fin

Types de variables manipulées

Remarques :

- Conflit de nom dans un sous-algorithme

Il ne faut pas donner le même nom à une variable locale et à un paramètre formel dans un même sous-algorithme,

→ Le sous-algorithme ne pourra plus les différencier.

- Nom identique entre sous-algorithmes

Il est possible de donner le même nom à des variables locales (ou paramètres formels) dans différents sous-algorithmes,

→ Elles peuvent avoir des contenus et types différents sans interférer entre elles.

- Masquage de variable globale

Si une variable locale porte le même nom qu'une variable globale,

→ La variable locale masque la variable globale dans le sous-algorithme concerné.

Types de variables manipulées – Exemple 1

Algorithme exemple 1

Var

a, b : Entier // variables globales

ProcédureafficherA ()

Début

Écrire("a vaut ", a) // Utilisation directe de 'a'

FinProcédure

Types de variables manipulées – Exemple 2

Procédure test()

Var

x : Entier // variable locale

Début

x $\leftarrow$ 5

Écrire(x)

FinProcédure

Types de variables manipulées – Exemple 3

Algorithme exemple

Variables

total : Entier // Variable globale accessible partout

Fonction double(x : Entier) : Entier // (paramètre formel x)

but

double $\leftarrow 2 * x$

retourner double

Fonction

Procédure ajouterValeur(val : Entier)

but

total $\leftarrow$ total + val

Procédure

Début

Var valeur : Entier // Variable locale au programme principal

total $\leftarrow$ 0 // initialisation au début du programme principal

Écrire("Entrez un entier :")

Lire(valeur)

ajouterValeur(valeur) // valeur : paramètre réel

Écrire("Le double de ", valeur, " est ", double(valeur))

Écrire("La somme totale est ", total)

FinAlgorithme

Exercice

Écrire les actions paramétrées (procédure ou fonction) permettant de résoudre les problèmes suivants :

1. Calculer la somme de deux nombres entiers.
2. Calculer la factorielle de N ($N!$) (Exemple : $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$)
3. Calculer le quotient et le reste de la division entière de deux nombres entiers A et B.
4. Vérifier si un caractère donné est une voyelle (voyelles : 'a', 'e', 'i', 'o', 'u', 'y').
5. Étant donné un entier A, calculer sa valeur absolue.